SweetSync — Technology Stack

Version: 1.0
Status: Draft
Last updated: May 7, 2026
Author: Raphael
Companion to: `sweetsync-prd.md`, `DESIGN.md`

Table of contents

Stack overview

Layer	Technology	Why
Mobile framework	Expo + React Native	Single codebase for iOS and Android. Familiar from prior projects.
Language	TypeScript	Type safety across the codebase.
Auth	Supabase Auth + Expo AuthSession	Google and Apple OAuth, JWT sessions, deep link preservation through auth flow.

Layer	Technology	Why
Database	Supabase (PostgreSQL)	Real-time subscriptions, Row Level Security, Storage, and Edge Functions in one platform.
AI — OCR	Gemini 2.5 Flash	Handles image and PDF input. Fast, cheap, multimodal.
AI — slot finding + activity suggestions	Gemini 2.5 Flash	Same model handles both tasks. Consistent API.
File storage	Supabase Storage	Keeps uploaded timetable files and auth in one platform. Signed URLs for Gemini.
Push notifications	Expo Push Notifications + FCM/APNs	Cross-platform, single API, free.
Notification triggers	Supabase Edge Functions + pg_cron	Server-side, event-driven, no separate backend needed.
Icons	Phosphor Icons (`phosphor-react-native`)	Clean, consistent, slightly rounded — fits the SweetSync personality.
Fonts	`@expo-google-fonts/fraunces` + `@expo-google-fonts/plus-jakarta-sans`	Display and body font pairing per the design system.

The entire backend runs on Supabase. No separate Node server, no additional hosting, no extra auth service. Fewer moving parts means fewer things to debug in early development.

Note on Deno: Supabase Edge Functions run on Deno, not Node.js. Most standard TypeScript works fine, but double-check any npm packages before use — some Node-specific packages don't run on Deno without a shim.

Authentication

Provider

Supabase Auth with Google and Apple OAuth via Expo AuthSession. Email and password signup is intentionally excluded — it creates unnecessary friction for invite-triggered users whose first instinct is to tap "Continue with Google."

Deep linking through auth

The most critical auth implementation detail in the whole app. When a user taps an invite link, signs in via OAuth, and completes the auth flow, the app must land them directly inside the invited room — not on the home screen.

```typescript
// Invite link structure
https://sweetsync.app/join?roomId=abc123&invitedBy=userId

// After OAuth completes, parse the redirect URL
// and route directly to the room
const roomId = url.searchParams.get('roomId');
if (roomId) router.push(`/room/${roomId}`);
```

If deep linking breaks and users land on an empty home screen, the motivation that drove them to download the app evaporates immediately. This is non-negotiable.

Session management

Supabase handles JWT refresh automatically. Tokens expire after one hour by default — keep that, do not extend it. Shorter token lifetimes are better security practice for an app handling personal schedule data.

```typescript
```

```tsx
// app/_layout.tsx
import { useFonts, Fraunces_700Bold, Fraunces_700Bold_Italic } from '@expo-google-fonts/frau
import { PlusJakartaSans_400Regular, PlusJakartaSans_600SemiBold } from '@expo-google-fonts/
import * as SplashScreen from 'expo-splash-screen';

SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
  const [fontsLoaded] = useFonts({
    Fraunces_700Bold,
    Fraunces_700Bold_Italic,
    PlusJakartaSans_400Regular,
    PlusJakartaSans_600SemiBold,
  });

  useEffect(() => {
    if (fontsLoaded) SplashScreen.hideAsync();
  }, [fontsLoaded]);

  if (!fontsLoaded) return null;
  return <Slot />;
}
```

Database

Platform

Supabase (PostgreSQL) with real-time subscriptions, Row Level Security, Edge Functions, and Storage.

Why Supabase earns its place

Three features specifically justify it for SweetSync:

Real-time subscriptions power the heat map updating live as members upload schedules, the "3 of 5 uploaded" progress indicator, and live vote counts during voting. All three are Supabase channels under the hood.

typescript

```
// Subscribe to schedule uploads in a room
const channel = supabase
  .channel(`room-${roomId}`)
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'schedules',
    filter: `room_id=eq.${roomId}`
  }, (payload) => {
    updateHeatMap(payload.new);
  })
  .subscribe();
```

Row Level Security is where most access control lives. Every table has RLS policies — members can only read data from rooms they belong to, and can only write their own data.

Storage handles uploaded timetable images and PDFs before they reach Gemini. Files go to Supabase Storage first, a signed URL is generated, and the Edge Function passes that URL to Gemini. API keys never touch the client.

Schema

sql

```
rooms             (id, name, host_id, created_at, session_status)
room_members      (room_id, user_id, joined_at)
schedules         (id, room_id, user_id, blocks jsonb, confirmed_at)
sessions          (id, room_id, status, voting_deadline)
slots             (id, session_id, start_time, end_time, ai_generated)
votes             (id, slot_id, user_id, response, created_at)
activities        (id, session_id, name, suggested_by, is_ai)
activity_votes    (id, activity_id, user_id, response)
events            (id, room_id, activity_id, slot_id, confirmed_at)
notifications_log (id, user_id, type, sent_at, room_id, send_at)
heat_map_cache    (id, room_id, grid jsonb, updated_at)
invites           (id, room_id, token, created_at, expires_at, used)
rate_limits       (id, user_id, action, count, window_start)
```

The `blocks jsonb` column on schedules stores extracted time blocks as structured JSON — flexible enough to handle different schedule formats without a rigid schema.

`heat_map_cache` stores precomputed heat map grids per room. Starts unused (client-side computation) and gets populated as scale demands server-side precomputation. Designed in from the start so the transition requires no schema change.

AI — OCR and schedule processing

Model

Gemini 2.5 Flash via Google AI Studio API (not Vertex AI). One model handles all three AI tasks in SweetSync:

1. **OCR extraction** — reads a timetable image or PDF and returns structured busy time blocks
2. **Slot finding** — cross-references all members' blocks and finds overlapping free windows
3. **Activity suggestions** — generates contextual activity ideas based on slot duration, group size, and optional location

Why not Vertex AI

Vertex AI has different auth, different billing, and a more complex setup. Google AI Studio's developer API is simpler to integrate with Expo and Supabase Edge Functions and is the right choice for a v1. Migrate to Vertex AI if you later need enterprise-grade SLAs, higher rate limits, or data residency guarantees.

Call flow

Files never go directly from client to Gemini. The flow is:

```
Client uploads file
→ Supabase Storage
→ Edge Function fetches signed URL
→ Edge Function calls Gemini with signed URL
→ Gemini returns extracted blocks
→ Edge Function writes blocks to schedules table
→ Client receives update via real-time subscription
```

Call volume per session

Action	Gemini calls
Schedule upload × 5 members	5 calls (OCR)
Slot finding	1 call
Activity suggestions	1 call
Total per session	**7 calls minimum**

At 40 active sessions per day that's 280+ calls — already past the free tier's 250 RPD limit. Enable billing before real users arrive.

Rate limiting on AI calls

Enforced in the Edge Function before the Gemini call fires. Five uploads per hour per user is a reasonable starting limit.

typescript

```typescript
async function checkRateLimit(userId: string, action: string): Promise<boolean> {
  const windowStart = new Date(Date.now() - 60 * 60 * 1000);
  const { count } = await supabase
    .from('rate_limits')
    .select('*', { count: 'exact' })
    .eq('user_id', userId)
    .eq('action', action)
    .gte('window_start', windowStart.toISOString());

  return (count ?? 0) < 5;
}
```

Handling 429 errors

Implement exponential backoff for Gemini rate limit errors. Queue and retry — never fail loudly to the user.

typescript

```typescript
async function callGeminiWithRetry(prompt: string, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      return await callGemini(prompt);
    } catch (err: any) {
      if (err.status === 429 && i < maxRetries - 1) {
        await new Promise(res => setTimeout(res, Math.pow(2, i) * 1000));
      } else throw err;
    }
  }
}
```

Security

Row Level Security — every table

RLS is the primary access control mechanism. No exceptions.

```sql
-- Members can only see schedules in their rooms
CREATE POLICY "room_member_select" ON schedules
  FOR SELECT USING (
    room_id IN (
      SELECT room_id FROM room_members
      WHERE user_id = auth.uid()
    )
  );

-- Members can only insert their own schedule
CREATE POLICY "own_schedule_insert" ON schedules
  FOR INSERT WITH CHECK (user_id = auth.uid());

-- Only hosts can update room settings
CREATE POLICY "host_room_update" ON rooms
  FOR UPDATE USING (host_id = auth.uid());
```

Without RLS, a motivated user could query any room's schedule data by guessing room IDs. That's a serious privacy violation for an app handling personal timetables.

Invite codes

Room invite links use short-lived signed tokens, not plain room IDs. Default expiry: 72 hours. Can be invalidated by the host after a session starts.

File validation

Before sending any uploaded file to Gemini, validate server-side:

- Accepted types: `image/jpeg`, `image/png`, `image/heic`, `application/pdf`
- Maximum size: 10MB
- MIME type checked from file bytes — never trust the client-provided content-type header

Environment variables

Variable	Location	Notes
`GEMINI_API_KEY`	Edge Function env only	Never in the client
`SUPABASE_SERVICE_ROLE_KEY`	Edge Function env only	Never in the client
`SUPABASE_ANON_KEY`	Client (safe to expose)	RLS enforces access control

Push notification permission timing

Request push permission immediately after a user's first schedule upload confirms successfully — not during onboarding. Asking when the app has just done something impressive dramatically improves acceptance rates.

Push notifications

Architecture

```
Database event
→ Supabase Edge Function (webhook or pg_cron)
→ Expo Push API
→ FCM (Android) / APNs (iOS)
→ Device
```

Notification schedule

Trigger	Message	Recipients
`room_members` INSERT	"You've been invited to [Room]"	New member
Member hasn't uploaded after 24hrs	"Your group is waiting on your schedule"	Non-uploaders
Host nudge action	"Don't forget to upload your schedule"	Specific member
All schedules confirmed	"Free slots are ready — go vote"	All members
All votes cast	"Results are in"	Host
`events` INSERT	"[Activity] confirmed — [Date] at [Time]"	All members

Trigger	Message	Recipients
Event T-24hrs	"Tomorrow: [Activity] at [Time]"	All members
Event T-1hr	"[Activity] starts in 1 hour"	All members

Scheduled notifications (pg_cron)

Time-based notifications use pg_cron running every 15 minutes. Pending notifications are stored in `notifications_log` with a `send_at` timestamp.

```sql
SELECT cron.schedule(
  'check-notifications',
  '*/15 * * * *',
  $$ SELECT fire_pending_notifications(); $$
);
```

Limits

Expo's push service handles 600 notifications per second per project — not a realistic constraint at SweetSync's scale. APNs and FCM delivery can have ±30 minute variance during high-traffic periods. Write notification copy that tolerates this — "starting soon" rather than "starting in exactly 1 hour."

Pricing — what's free and what isn't

Supabase

Plan	Cost	Key limits
Free	$0/month	500MB database, 1GB storage, 50K MAUs, 500K Edge Function invocations. Projects pause after 7 days of inactivity.
Pro	$25/month base	8GB database, 100GB storage, 100K MAUs. No project pausing. Daily backups.

The free tier is development-only. Move to Pro before the first real user.

Gemini API

Model	Free tier limits	Paid tier
Gemini 2.5 Flash	10 RPM, 250 RPD	$0.10/million input tokens (Flash-Lite)
Gemini 2.5 Flash-Lite	15 RPM, 1,000 RPD	Cheapest production-ready option
Gemini 2.5 Pro	5 RPM, 100 RPD	Too restricted for production use

On the free tier, Google may use API inputs and outputs to improve their models. For an app processing personal university timetables, enable billing before real users arrive to opt out of data sharing.

Free tier limits were reduced 50-80% in December 2025. Treat it as prototyping-only.

Expo push notifications

Completely free. No cost per notification. No limits relevant to SweetSync's volume.

EAS Build (cloud builds) has a free tier with limited monthly builds — sufficient for development. Paid plans start if you need frequent CI/CD builds.

Store accounts

Platform	Cost	Type
Apple Developer Program	$99/year	Required for iOS App Store distribution
Google Play Developer	$25 one-time	Required for Android Play Store distribution

Realistic monthly cost

Stage	Supabase	Gemini	Expo	Total/month
Development	$0	$0	$0	**$0**
Early users (< 40 sessions/day)	$25	~$1–5	$0	**~$26–30**
Growing (hundreds of sessions/day)	$25 + compute	~$10–30	$0	**~$35–55**

One-time: Apple Developer $99/year, Google Play $25.

Scalability

Supabase

Scales cleanly from development through tens of thousands of users without architectural changes — only compute size changes.

Real-time connections are the area to watch. Always clean up subscriptions on component unmount.

```typescript
useEffect(() => {
  const channel = supabase.channel(`room-${roomId}`).subscribe();
  return () => { supabase.removeChannel(channel); };
}, [roomId]);
```

Read replicas and PgBouncer are available on higher plans for heavy read workloads. Not needed at v1 but available without migrating off the platform.

Edge Function cold start latency is acceptable for background triggers. Keep user-facing fast-response logic in the client or in database functions.

Gemini API

The most constrained layer at scale. Call volume per session is 7+ calls minimum. Paid tier scales linearly and stays cheap — even at 1,000 sessions per day, Gemini costs stay well under $50/month.

The spike risk is simultaneous uploads from large groups. Implement a request queue with exponential backoff in Edge Functions so burst traffic throttles gracefully rather than failing loudly.

Heat map computation

Starts client-side. Move server-side when needed.

```typescript
```

```
// Heat map shade calculation — isolated function, moves server-side without redesign
function getHeatShade(freeCount: number, totalMembers: number): string {
  const ratio = freeCount / totalMembers;
  if (ratio === 1)   return '#FAFAF9';
  if (ratio >= 0.8)  return '#EEEDFE';
  if (ratio >= 0.6)  return '#AFA9EC';
  if (ratio >= 0.4)  return '#7F77DD';
  return '#534AB7';
}
```

When scale demands it, an Edge Function recomputes the heat map on each schedule confirmation and writes to `heat_map_cache`. The client fetches the cached result. Cache invalidation: any new schedule confirmation triggers a recalculation.

Notifications at scale

pg_cron handles tens of thousands of pending notifications cleanly. Beyond hundreds of thousands, move to a managed job queue (BullMQ or similar). Not a v1 concern.

Scaling roadmap

Stage 1 — Development and testing

- Everything on free tiers
- Client-side heat map computation
- Cost: **$0**

Stage 2 — Early users

Hundreds of active users, tens of active rooms per day.

- Supabase Pro — removes project pausing, adds daily backups
- Gemini billing enabled — removes data-sharing, accesses higher rate limits
- Client-side heat map still fine
- Cost: **~$25–35/month**

Stage 3 — Growth

Thousands of active users, hundreds of rooms per day.

- Supabase compute upgrade (dedicated instance)

- Heat map computation moves server-side (`heat_map_cache`)
- Gemini request queue added in Edge Functions
- Aggressive Supabase Realtime channel cleanup
- Cost: ~**$50–150/month**

Stage 4 — Scale

Tens of thousands of active users.

- Supabase read replicas for heavy read workloads
- Evaluate Vertex AI over Google AI Studio for SLAs and data residency
- Managed job queue for notification scheduling
- CDN caching for static assets and infrequently-changing room data

---

Implementation notes

Theme constants

```typescript
```

```typescript
// constants/theme.ts
export const colors = {
  peachBase:     '#FDF6F0',
  peachSoft:     '#F5C4B3',
  peachMid:      '#F0997B',
  peachPunch:    '#D85A30',
  peachDeep:     '#712B13',
  indigoBase:    '#EEEDFE',
  indigoSoft:    '#AFA9EC',
  indigoMid:     '#7F77DD',
  indigoPunch:   '#534AB7',
  indigoDeep:    '#3C3489',
  mintBase:      '#E1F5EE',
  mintSoft:      '#9FE1CB',
  mintPunch:     '#0F6E56',
  pageBg:        '#FAFAF9',
  surface:       '#FFFFFF',
  textPrimary:   '#1A1A1A',
  textSecondary: '#6B6B6B',
  textTertiary:  '#A0A0A0',
  borderDefault: 'rgba(0,0,0,0.08)',
} as const;

export const fonts = {
  display:       'Fraunces_700Bold',
  displayItalic: 'Fraunces_700Bold_Italic',
  body:          'PlusJakartaSans_400Regular',
  bodySemibold:  'PlusJakartaSans_600SemiBold',
} as const;

export const spacing = {
  1: 4, 2: 8, 3: 12, 4: 16,
  5: 20, 6: 24, 8: 32, 10: 40,
} as const;

export const radius = {
  sm: 8, md: 12, lg: 16, xl: 24, full: 999,
} as const;
```

Reusable card component

typescript

```tsx
// components/Card.tsx
import { View, StyleSheet } from 'react-native';
import { colors, spacing, radius } from '@/constants/theme';

type CardVariant = 'peach' | 'indigo' | 'mint' | 'neutral';

const cardStyles: Record<CardVariant, { background: string; borderColor: string }> = {
  peach:   { background: colors.peachBase,  borderColor: colors.peachSoft  },
  indigo:  { background: colors.indigoBase, borderColor: colors.indigoSoft },
  mint:    { background: colors.mintBase,   borderColor: colors.mintSoft   },
  neutral: { background: '#F7F6F3',         borderColor: colors.borderDefault },
};

export function Card({
  variant = 'neutral',
  children,
  style,
}: {
  variant?: CardVariant;
  children: React.ReactNode;
  style?: object;
}) {
  const { background, borderColor } = cardStyles[variant];
  return (
    <View style={[styles.card, { backgroundColor: background, borderColor }, style]}>
      {children}
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    borderRadius: radius.lg,
    borderWidth: 0.5,
    padding: spacing[4],
  },
});
```

Animation constants

typescript

```ts
// constants/animation.ts
import { Easing } from 'react-native-reanimated';

export const transitions = {
  cardEntrance:     { duration: 280, easing: Easing.out(Easing.cubic) },
  buttonPress:      { duration: 120, easing: Easing.in(Easing.cubic) },
  screenPush:       { duration: 320, easing: Easing.inOut(Easing.cubic) },
  confirmedPop:     { duration: 400, damping: 14, stiffness: 180 },
  listStaggerDelay: 50,
} as const;
```

Pricing figures verified as of May 2026. Re-verify before production launch — free tier limits and pricing change frequently, particularly for the Gemini API.